SHOPRAVEN TECHNICAL REFERENCE MANUAL • ANTIGRAVITY IDE INJECTION BLUEPRINT

ShopRaven — Comprehensive System Architecture & Codebase Manual

Exhaustive system reference manual documenting every single file, class, function, call graph relationship, database schema, pricing formula, and operational flow across the ShopRaven Autonomous E-Commerce AI Cofounder platform for **nnstore.shop**.

Total Files Documented	Architectural Modules	Alippo Parity Features	Primary Storefront & Suppliers	Document Edition
122 Files (Python + Node.js + WP Plugin + React/HTML)	**24 Sections** (Full-Stack Autonomous Pipeline)	**13 / 13 Features** (100% Parity + PK COD Upgrades)	**nnstore.shop** (WooCommerce) • **HHC** & **Oshi Dropshipping**	**v1.0.0 Enterprise** • October 2026 (Naeem Ashiq / NaeeemJatt)

Target Systems: Python 3.11+ (FastAPI • SQLAlchemy 2.0 • Celery • Playwright • BeautifulSoup4 • Pillow • Gemini 2.5 Flash Structured Outputs) • Node.js 18+ (HTTP/Webhook Orchestrator • Cron Scheduler • WooCommerce REST API v3 Client • Discord Rich Embed Bot • WhatsApp Cloud API) • WordPress Plugin (`shopraven-bridge`) • Interactive Control Room Dashboard.

Business Rules Locked: Storefront URL: https://nnstore.shop • Suppliers: HHC Dropshipping & Oshi Dropshipping (authenticated session scraping) • Profit Target: **+40% to +50% net profit** over supplier wholesale cost + **Rs. 150 flat separate COD delivery** • Price Rounding: *49 or *99 psychological endings • Notification Channel: **Discord Rich Embeds**.

Table of Contents — Architectural Specification Map

Section	Architectural Subsystem / Specification Chapter	Primary Layer & Scope
01	Executive Architecture & 6-Phase End-to-End Autonomous Workflow Specification	Platform Core & Research Logic
02	Complete Project File System Tree (`NaeeemJatt/ShopRaven`)	Directory Structure (86 Files)
03	Project Root Configuration, Docker & Environment	7 Files (`.env`, Docker, CI)
04	Python Backend Core, FastAPI Entrypoint & Celery Task Queue	8 Files (`backend/app/`)
05	Database Models & SQLAlchemy 2.0 ORM Schemas	7 Files (`backend/app/models/`)
06	Pydantic v2 Validation Schemas & Structured Output Contracts	5 Files (`backend/app/schemas/`)
07	Pakistani Market & Search Demand Scout Engine	5 Files (`backend/app/scout/`)
08	Authenticated Dual-Supplier Vault (HHC & Oshi Dropshipping)	5 Files (`backend/app/suppliers/`)
09	Competitive Price Researcher & +40%–50% Margin Guard Engine	4 Files (`backend/app/pricing/`)
10	Automated 1080×1080 Product Image Studio & WP Media Uploader	4 Files (`backend/app/studio/`)
11	SEO, AEO (Answer Engine Optimization) & JSON-LD Schema Engine	4 Files (`backend/app/seo_aeo/`)
12	Roman-Urdu & Misspelling Search Synonym Mapper (Alippo Feature #4)	3 Files (`backend/app/seo_aeo/`)
13	Deal Hunter Price-Hesitation Coupon & Margin-Floor Bot (Alippo Feature #5)	3 Files (`backend/app/coupons/`)
14	Autonomous SEO Blog Generator with Embedded Product Cards (Alippo Feature #7)	3 Files (`backend/app/blog/`)
15	30-Day AI Sales Forecaster & True Net Profit Accounting (Alippo #2 & #13)	4 Files (`backend/app/forecasting/`)
16	Python FastAPI Routers & Celery Autonomous Pipeline Tasks	8 Files (`backend/app/api/`)
17	Node.js Orchestrator Core, Cron Scheduler & WooCommerce Client	8 Files (`orchestrator/src/`)
18	Storefront Automation: Menu Reorder, 301 Redirects & Restock Waitlist (Alippo #6, #8, #9)	4 Files (`orchestrator/src/woocommerce/`)
19	COD-to-Prepaid Split, RTO Risk Scorer & WhatsApp Cart Recovery (Alippo #11 & #12)	4 Files (`orchestrator/src/whatsapp/`)
20	Discord Rich Embed Notifier & 1-Click Publish Slash Commands	3 Files (`orchestrator/src/discord/`)
21	WordPress / WooCommerce Plugin for `nnstore.shop` (Alippo #8, #10, #11, #13)	5 Files (`wordpress-plugin/`)
22	ShopRaven Control Room Frontend & Analytics Dashboard	8 Files (`frontend/`)
23	Complete Alippo 13-Feature Parity Matrix & Mathematical Formulas	Cross-Platform Reference

Section	Architectural Subsystem / Specification Chapter	Primary Layer & Scope
24	Antigravity IDE Master Injection Prompt & Step-by-Step Build Sequence	Execution Playbook

Section 1: Executive Architecture & End-to-End Autonomous Workflow Specification

ShopRaven operates as a unified autonomous e-commerce operations, product research, dual-supplier sourcing, dynamic pricing, SEO/AEO publishing, and real-time analytics platform built specifically for nnstore.shop in the Pakistani Cash-on-Delivery (COD) market. The platform is structured around four synchronized pipelines:

A. Autonomous Pakistani Market Research & Catalog Gap Pipeline (Every 6 Hours)

1. **Multi-Source Demand Ingestion:** `SearchDemandScanner` queries Google Autocomplete Pakistan (`gl=pk`), Google Trends Pakistan, Daraz Pakistan category movers, PriceOye gadget leaders, and active Pakistani ads in the Meta Ad Library across all 7 official `nnstore.shop` categories (*Beauty & Care, Kitchen Accessories, Home & Living, Gym Accessories, Electronic Accessories, Auto & Tools, Accessories*).
2. **Live Catalog Gap Analysis:** `CatalogGapChecker` fetches all live and draft products from `https://nnstore.shop/wp-json/wc/v3/products` and computes token-level similarity against every trending search keyword.
3. **Intelligent Routing:** If the product already exists on `nnstore.shop`, ShopRaven logs `ALREADY_IN_STORE` on Discord and queues an SEO/AEO tag refresh with the newly discovered keywords. If the product is missing, ShopRaven broadcasts a purple `NEW_PRODUCT_OPPORTUNITY` embed to Discord and triggers Pipeline B.

B. Dual-Supplier Authenticated Sourcing & +40%–50% Competitive Pricing Pipeline

1. **Zero-Friction Session Vault:** `HHCDropshippingClient` (member.hhcdropshipping.com) and `OshiDropshippingClient` (portal.oshidropship.com) authenticate using credentials from `.env` and cache session cookies in Redis.
2. **Head-to-Head Wholesale Comparison:** `DualSupplierComparator` searches both portals simultaneously, filters out out-of-stock items, compares the wholesale PKR cost, selects the lower-cost supplier, and downloads raw descriptions, specifications, and up to 5 high-resolution images.
3. **Landed COD Competitive Price Calculation:** `CompetitivePricingEngine` scrapes live Pakistani retail prices for the item to find the Market Average (M_{avg}). Because `nnstore.shop` charges a flat **Rs. 150** delivery fee separately at checkout while competitors charge **Rs. 200–250**, the engine compares total doorstep landed cost:

   ```
   Competitor Landed Total = M_avg + Rs. 220
   NN Store Landed Total   = Sale_Price + Rs. 150
   Target Profit Band      = [Wholesale_Cost * 1.40, Wholesale_Cost * 1.50]
   ```

   The engine locks in up to +50% profit when the market average allows, positions just below competitor landed cost inside the +40%–50% band when competitive, never drops below the +40% profit floor, and rounds to `*49` or `*99` (e.g., `Rs. 1,399` with strikethrough Regular Price `Rs. 2,049`).

C. Studio Image Processing, Localized Roman-Urdu SEO/AEO & WooCommerce Draft Pipeline

1. **1080×1080 Studio Image Standardization:** `ProductImageStudio` centers raw supplier photos on a clean `1080x1080` studio square canvas, boosts contrast (+6%) and sharpness (+12%), and uploads them to WordPress Media Library with localized Pakistani SEO `alt` attributes.
2. **Roman-Urdu Search Mapping & AEO JSON-LD:** `NNStoreSEOAEOEngine` generates `nnstore.shop`'s signature short description (1-line bold hook + 4 concise bullets), full persuasive HTML, 15–20 Roman-Urdu & typo synonyms, RankMath/Yoast meta tags, a 45-word AEO Direct Answer summary, 3 buyer FAQs (covering PKR price, 3–5 day nationwide COD for Rs. 150, and WhatsApp support at `+92 312 4999196`), and valid `Product` + `OfferShippingDetails` + `FAQPage` JSON-LD schema.
3. **WooCommerce Draft Creation & 1-Click Discord Publish:** `WooCommerceProductPublisher` creates the product in `draft` status on `nnstore.shop` and posts a Discord embed with a 1-click `/publish` command and direct WP Admin edit link.

D. Storefront Conversion, COD/RTO Protection & Executive Control Room Pipeline

1. **On-Site Conversion Boosters (WordPress Plugin):** Renders the Converting Cart Progress Bar (*"Add Rs. X more for Free Delivery!"*), Deal Hunter exit-intent coupon bot (capped at the +40% profit floor), Out-of-Stock WhatsApp waitlist capture, and 24/7 Product Q&A chatbot.
2. **Pakistani COD Confirmation & Abandoned Cart Recovery:** Sends automated WhatsApp messages for abandoned carts and new COD orders, offering to waive the Rs. 150 shipping fee if the customer switches to prepaid (Raast / EasyPaisa / JazzCash) to slash RTO return losses.

Section 2: Complete Project File System Tree (86 Files)

The following directory tree defines the complete repository layout for `NaeeemJatt/ShopRaven`:

```
ShopRaven/
├── .env.example
├── .gitignore
├── Makefile
├── README.md
├── docker-compose.yml
├── skills-lock.json
├── .github/
│   └── workflows/
│       └── ci.yml
├── backend/
```

```
│   ├── Dockerfile
│   ├── alembic.ini
│   ├── requirements.txt
│   ├── alembic/
│   │   ├── env.py
│   │   └── versions/
│   │       ├── 001_create_trend_keywords.py
│   │       ├── 002_create_supplier_offers.py
│   │       ├── 003_create_pricing_audits.py
│   │       ├── 004_create_wc_products_sync.py
│   │       ├── 005_create_restock_waitlists.py
│   │       ├── 006_create_customer_segments.py
│   │       └── 007_create_analytics_snapshots.py
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   ├── config.py
│   │   ├── database.py
│   │   ├── celery_app.py
│   │   ├── models/
│   │   │   ├── trend_keyword.py
│   │   │   ├── supplier_offer.py
│   │   │   ├── pricing_audit.py
│   │   │   ├── wc_product_sync.py
│   │   │   ├── restock_waitlist.py
│   │   │   ├── customer_segment.py
│   │   │   └── analytics_snapshot.py
│   │   ├── schemas/
│   │   │   ├── research_schema.py
│   │   │   ├── supplier_schema.py
│   │   │   ├── pricing_schema.py
│   │   │   ├── seo_aeo_schema.py
│   │   │   └── analytics_schema.py
│   │   ├── scout/
│   │   │   ├── __init__.py
│   │   │   ├── google_trends_pk.py
│   │   │   ├── daraz_priceoye_movers.py
│   │   │   ├── meta_ads_pk_verifier.py
│   │   │   └── demand_scanner.py
│   │   ├── suppliers/
│   │   │   ├── __init__.py
│   │   │   ├── session_vault.py
│   │   │   ├── hhc_client.py
│   │   │   ├── oshi_client.py
│   │   │   └── comparator.py
│   │   ├── pricing/
│   │   │   ├── __init__.py
│   │   │   ├── competitor_scraper.py
│   │   │   ├── rounding.py
│   │   │   └── engine.py
│   │   ├── studio/
│   │   │   ├── __init__.py
│   │   │   ├── image_processor.py
│   │   │   ├── badge_overlay.py
│   │   │   └── wp_media_uploader.py
│   │   ├── seo_aeo/
│   │   │   ├── __init__.py
│   │   │   ├── prompts.py
│   │   │   ├── json_ld_builder.py
│   │   │   ├── roman_urdu_dictionary.json
│   │   │   ├── roman_urdu_mapper.py
│   │   │   └── generator.py
│   │   ├── coupons/
│   │   │   ├── __init__.py
│   │   │   └── deal_hunter.py
│   │   ├── blog/
│   │   │   ├── __init__.py
│   │   │   ├── auto_blogger.py
│   │   │   └── product_card_template.py
│   │   ├── forecasting/
│   │   │   ├── __init__.py
│   │   │   ├── true_profit_accountant.py
│   │   │   ├── sales_predictor.py
│   │   │   └── monthly_report.py
│   │   ├── api/
│   │   │   ├── router.py
│   │   │   └── routes/
│   │   │       ├── research.py
│   │   │       ├── suppliers.py
│   │   │       ├── pricing.py
│   │   │       ├── seo_aeo.py
│   │   │       ├── blog.py
│   │   │       └── analytics.py
```

```
│   │   └── tasks/
│   │       └── pipeline_tasks.py
│   └── tests/
│       ├── conftest.py
│       ├── test_roman_urdu_mapper.py
│       ├── test_deal_hunter.py
│       ├── test_pricing_engine.py
│       └── test_seo_aeo_schema.py
├── orchestrator/
│   ├── Dockerfile
│   ├── package.json
│   ├── src/
│   │   ├── server.js
│   │   ├── config/
│   │   │   └── env.js
│   │   ├── jobs/
│   │   │   └── cron_scheduler.js
│   │   ├── woocommerce/
│   │   │   ├── wc_api_client.js
│   │   │   ├── catalog_sync.js
│   │   │   ├── product_publisher.js
│   │   │   ├── menu_reorder.js
│   │   │   ├── smart_redirects.js
│   │   │   ├── restock_notifier.js
│   │   │   └── coupon_service.js
│   │   ├── whatsapp/
│   │   │   ├── cloud_client.js
│   │   │   ├── cart_recovery.js
│   │   │   └── cod_rto_guard.js
│   │   └── discord/
│   │       ├── notifier.js
│   │       ├── bot_commands.js
│   │       └── embed_templates.js
│   └── tests/
│       └── orchestrator.test.js
├── wordpress-plugin/
│   └── shopraven-bridge/
│       ├── shopraven-bridge.php
│       ├── includes/
│       │   ├── class-cart-progress-bar.php
│       │   ├── class-prepaid-incentive.php
│       │   └── class-product-qa-bot.php
│       └── assets/
│           └── shopraven-storefront.js
└── frontend/
    ├── nnstore_ai_dashboard.html
    └── src/
        ├── App.tsx
        ├── store/
        │   └── useShopRavenStore.ts
        └── components/
            ├── ExecutiveAnalyticsView.tsx
            ├── KeywordGapFinderView.tsx
            ├── SupplierPriceSimulatorView.tsx
            ├── SEOAEOQueueView.tsx
            └── DiscordLiveFeedView.tsx
```

Section 3: Project Root Configuration, Docker & Environment (7 Files)

Root-level configuration, environment variable templates, Docker Compose service definitions, and master repository documentation required to bootstrap ShopRaven in local development or production.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
`.env.example` 94 lines • 3,840 bytes	Master environment template defining all required secrets and runtime flags for the Python AI Engine, Node.js Orchestrator, WordPress/WooCommerce REST API v3 (nnstore.shop), HHC Dropshipping, Oshi Dropshipping, Discord Webhook/Bot, WhatsApp Cloud API, and Gemini 2.5 Flash.	`WC_STORE_URL, WC_CONSUMER_KEY, WC_CONSUMER_SECRET` • `HHC_EMAIL, HHC_PASSWORD, OSHI_EMAIL, OSHI_PASSWORD` • `PROFIT_MIN_PCT=0.40, PROFIT_MAX_PCT=0.50, DELIVERY=150` • `DISCORD_WEBHOOK_URL, WHATSAPP_TOKEN, GEMINI_API_KEY`	**Called By:** backend/app/config.py, orchestrator/src/config/env.js, docker-compose.yml **Deps:** `python-dotenv, dotenv (Node.js)`
`docker-compose.yml` 118 lines • 4,120 bytes	Multi-container orchestration launching PostgreSQL 16 (analytics, keyword history, price logs, waitlists), Redis 7 (Celery broker & session cache), Python FastAPI + Celery Worker (shopraven-backend), Node.js Orchestrator & Discord Bot (shopraven-orchestrator), and Nginx reverse proxy.	`service: postgres (port 5432)` • `service: redis (port 6379)` • `service: shopraven-backend (port 8000)` • `service: shopraven-celery-worker` • `service: shopraven-orchestrator (port 4000)`	**Called By:** Top-level deployment entrypoint (docker compose up -d --build) **Deps:** `Docker Engine 24+, Docker Compose v2`

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
README.md 310 lines • 14,890 bytes	Master platform documentation detailing system architecture, the 6-phase Pakistani market research loop, the 13 Alippo AI Cofounder parity modules, and quickstart commands.	None (documentation file)	**Called By:** Developer & Antigravity IDE onboarding context **Deps:** Markdown
.gitignore 74 lines • 960 bytes	Excludes .env secrets, Python __pycache__, virtual environments, Node.js node_modules, Playwright browser caches, and temporary studio image renders from Git version control.	None (Git configuration)	**Called By:** Git CLI **Deps:** None
Makefile 62 lines • 1,950 bytes	Developer automation targets for starting services, running Alembic migrations, triggering a manual research cycle, running pytest and Jest suites, and syncing WooCommerce categories.	target: make setup • target: make dev • target: make migrate • target: make scan-now • target: make test	**Called By:** Developer CLI & GitHub Actions CI **Deps:** GNU Make
.github/workflows/ci.yml 96 lines • 3,210 bytes	GitHub Actions CI pipeline for NaeeemJatt/ShopRaven executing Ruff lint, Black formatting, Alembic migration verification, pytest unit/integration tests, and Node.js orchestrator syntax checks.	job: backend-lint-and-test • job: orchestrator-test • job: frontend-build-verify	**Called By:** GitHub Actions on push/PR to main and improvement branches **Deps:** actions/checkout@v4, actions/setup-python@v5, actions/setup-node@v4
skills-lock.json 38 lines • 1,420 bytes	Locks AI agent prompt templates, schema versions, and scraper selector version hashes.	version: 1.0.0 • schema_lock_hash	**Called By:** backend/app/config.py **Deps:** Standard JSON

Section 4: Python Backend Core, FastAPI Entrypoint & Celery Queue (8 Files)

Core Python application layer (backend/) providing the FastAPI REST server, configuration management, SQLAlchemy async database session factory, and Celery distributed worker queue.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/requirements.txt 42 lines • 980 bytes	Pinned Python dependencies for the ShopRaven backend and AI worker engine.	fastapi==0.115.0 • uvicorn[standard]==0.30.6 • sqlalchemy==2.0.35 • alembic==1.13.2 • celery==5.4.0 • redis==5.0.8 • requests==2.32.3 • beautifulsoup4==4.12.3 • playwright==1.47.0 • Pillow==10.4.0 • google-genai==0.3.0 • pydantic==2.9.2	**Called By:** backend/Dockerfile, Makefile **Deps:** pip
backend/Dockerfile 44 lines • 1,240 bytes	Container image build specification with Python 3.11-slim, libjpeg/zlib for Pillow, and Playwright Chromium.	FROM python:3.11-slim • CMD uvicorn app.main:app	**Called By:** docker-compose.yml **Deps:** Docker
backend/app/__init__.py 4 lines • 92 bytes	Initializes the ShopRaven backend Python package and exposes __version__ = '1.0.0'.	__version__	**Called By:** backend/app/main.py **Deps:** Standard library
backend/app/config.py 112 lines • 4,180 bytes	Centralized Pydantic BaseSettings configuration loader reading .env variables, validating margin rules (+40% to +50% profit, Rs. 150 separate COD shipping), and exposing typed settings singleton.	class Settings(BaseSettings) • fn: get_settings() -> Settings • const: NNSTORE_CATEGORIES (7 official categories)	**Called By:** backend/app/main.py, backend/app/suppliers/hhc_client.py, backend/app/suppliers/oshi_client.py, backend/app/pricing/engine.py, backend/app/seo_aeo/generator.py **Deps:** pydantic_settings, os, functools.lru_cache
backend/app/database.py 68 lines • 2,190 bytes	SQLAlchemy 2.0 declarative base, engine pool, and FastAPI dependency injection get_db() generator.	class Base(DeclarativeBase) • engine = create_engine() • SessionLocal = sessionmaker() • fn: get_db() -> Generator[Session, None, None]	**Called By:** backend/app/main.py, backend/app/api/routes/*.py, backend/app/tasks/*.py **Deps:** sqlalchemy, app.config
backend/app/main.py 134 lines • 4,890 bytes	FastAPI application entrypoint mounting CORS middleware, health check endpoints, and all modular API routers (/api/v1/research, /api/v1/suppliers, /api/v1/pricing, /api/v1/seo-aeo, /api/v1/blog, /api/v1/analytics, /api/v1/forecast).	app = FastAPI(title='ShopRaven Engine') • fn: lifespan(app: FastAPI) • route: GET /health	**Called By:** uvicorn server, backend/tests/conftest.py **Deps:** fastapi, app.api.router, app.database, app.config

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/celery_app.py 58 lines • 1,890 bytes	Celery distributed worker and Beat scheduler configuration for asynchronous scraping, image rendering, weekly SEO blog generation, and 30-day sales forecasting.	`celery_app = Celery('shopraven')` • `beat_schedule: run_market_scout_every_6h, run_stock_sync_every_2h, run_weekly_blog_and_menu_sort`	**Called By:** backend/app/tasks/pipeline_tasks.py **Deps:** `celery, redis, app.config`
backend/alembic.ini 48 lines • 1,410 bytes	Alembic schema migration configuration pointing to backend/alembic/versions/.	`script_location = alembic`	**Called By:** Alembic CLI **Deps:** `alembic, sqlalchemy`

Section 5: Database Models & SQLAlchemy 2.0 ORM Schemas (7 Files)

Persistent PostgreSQL tables tracking discovered Pakistani trends, HHC & Oshi supplier offers, competitor price history, WooCommerce product sync state, out-of-stock WhatsApp waitlists, and store analytics.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/models/trend_keyword.py 76 lines • 2,640 bytes	Stores trending Pakistani search keywords, search volume velocity, source (Google PK, Daraz, Meta Ads), mapped NN Store category, and catalog gap check status (IN_STORE vs MISSING_OPPORTUNITY).	`class TrendKeyword(Base)` — `table: trend_keywords` • `columns: id, keyword, normalized_tokens, category_name, monthly_searches, growth_pct, source, exists_in_store, matched_wc_product_id, last_scanned_at`	**Called By:** backend/app/scout/demand_scanner.py, backend/app/api/routes/research.py **Deps:** `sqlalchemy, app.database.Base`
backend/app/models/supplier_offer.py 88 lines • 3,120 bytes	Stores wholesale product offers scraped from HHC Dropshipping and Oshi Dropshipping, including SKU, wholesale cost in PKR, stock status, raw descriptions, specifications JSON, and raw image URLs.	`class SupplierOfferRecord(Base)` — `table: supplier_offers` • `columns: id, trend_keyword_id, supplier_name ('HHC' | 'OSHI'), supplier_url, supplier_sku, product_title, wholesale_cost_pkr, in_stock, raw_description, specs_json, image_urls_json, scraped_at`	**Called By:** backend/app/suppliers/comparator.py, backend/app/tasks/stock_sync.py **Deps:** `sqlalchemy, app.database.Base`
backend/app/models/pricing_audit.py 82 lines • 2,890 bytes	Audits every price calculation performed by CompetitivePricingEngine, storing wholesale cost, +40% floor, +50% target, scraped Pakistani competitor prices, landed COD comparison (Rs. 150 vs Rs. 220), and final *49/*99 sale & regular prices.	`class PricingAudit(Base)` — `table: pricing_audits` • `columns: id, supplier_offer_id, wholesale_cost_pkr, min_40_pkr, max_50_pkr, competitor_prices_json, market_avg_pkr, recommended_sale_pkr, recommended_regular_pkr, net_profit_pkr, profit_margin_pct, strategy_note, created_at`	**Called By:** backend/app/pricing/engine.py, backend/app/api/routes/pricing.py **Deps:** `sqlalchemy, app.database.Base`
backend/app/models/wc_product_sync.py 94 lines • 3,480 bytes	Tracks every product synced to or created on https://nnstore.shop, including WooCommerce ID, status ('draft' | 'publish' | 'outofstock' | 'redirected'), RankMath SEO score, JSON-LD schema, views, orders, conversion velocity, and 301 redirect target URL.	`class WCProductSync(Base)` — `table: wc_products_sync` • `columns: id, wc_product_id, title, slug, category_name, supplier_name, supplier_sku, wholesale_cost_pkr, sale_price_pkr, regular_price_pkr, net_profit_pkr, status, menu_order, total_views, total_orders, conversion_rate, redirect_target_url, json_ld_schema`	**Called By:** orchestrator/src/woocommerce/catalog_sync.js, backend/app/seo_aeo/generator.py **Deps:** `sqlalchemy, app.database.Base`

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/models/restock_waitlist.py 64 lines • 2,150 bytes	Implements Alippo Feature #8 (Out-of-Stock 'Notify Me on WhatsApp'). Stores customer WhatsApp numbers waiting for an out-of-stock product on nnstore.shop and tracks 1-tap checkout link dispatch.	`class RestockWaitlist(Base) — table: restock_waitlists • columns: id, wc_product_id, product_title, customer_whatsapp, notified, checkout_link_sent_at`	**Called By:** orchestrator/src/woocommerce/restock_notifier.js **Deps:** `sqlalchemy, app.database.Base`
backend/app/models/customer_segment.py 78 lines • 2,740 bytes	Implements Alippo Feature #5 & #12 (Customer Segmentation & Deal Hunter Tracking). Segments shoppers into NEW_VISITOR, RETURNING_VIP, DEAL_HUNTER, Cart Abandoner, or HIGH_RTO_RISK.	`class CustomerSegment(Base) — table: customer_segments • columns: id, phone_or_email, city, total_orders, delivered_orders, rto_orders, total_spent_pkr, segment_tag, last_coupon_code, updated_at`	**Called By:** backend/app/coupons/deal_hunter.py, orchestrator/src/whatsapp/cart_recovery.js **Deps:** `sqlalchemy, app.database.Base`
backend/app/models/analytics_snapshot.py 86 lines • 3,090 bytes	Daily time-series rollups for the ShopRaven Control Room Dashboard: total traffic, new vs. returning visitors, gross sales PKR, net realized profit PKR, COD delivered vs in-transit vs RTO counts, city-wise orders, and AEO citations.	`class AnalyticsSnapshot(Base) — table: analytics_snapshots • columns: id, snapshot_date, total_visitors, new_visitors, returning_visitors, gross_sales_pkr, shipping_collected_pkr, supplier_cost_pkr, net_profit_pkr, total_orders, delivered_orders, in_transit_orders, rto_orders, aeo_referrals, city_breakdown_json`	**Called By:** backend/app/api/routes/analytics.py, backend/app/forecasting/sales_predictor.py **Deps:** `sqlalchemy, app.database.Base`

Section 6: Pydantic v2 Validation Schemas & Structured Output Contracts (5 Files)

Strict Pydantic v2 schemas enforcing type safety across REST API requests, Gemini 2.5 Flash Structured Outputs, and Node.js-to-Python webhook payloads.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/schemas/research_schema.py 62 lines • 1,980 bytes	Request and response schemas for Pakistani market trend discovery and keyword velocity.	`class TrendItemSchema(BaseModel)` • `class MarketScanRequest(BaseModel)` • `class MarketScanResponse(BaseModel)`	**Called By:** backend/app/api/routes/research.py **Deps:** `pydantic`
backend/app/schemas/supplier_schema.py 74 lines • 2,420 bytes	Normalized schema for HHC Dropshipping and Oshi Dropshipping product offers and comparison results.	`class SupplierOfferSchema(BaseModel)` • `class DualSupplierCompareResponse(BaseModel)`	**Called By:** backend/app/suppliers/comparator.py, backend/app/api/routes/suppliers.py **Deps:** `pydantic`

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/schemas/pricing_schema.py 68 lines • 2,250 bytes	Input/output contract for the +40%-50% Competitive Pricing Engine with Rs. 150 COD landed cost math.	`class PricingRequestSchema(BaseModel)` • `class CompetitivePricingResponseSchema(BaseModel)`	**Called By:** backend/app/pricing/engine.py, backend/app/api/routes/pricing.py **Deps:** `pydantic`
backend/app/schemas/seo_aeo_schema.py 96 lines • 3,540 bytes	Enforces Gemini Structured Outputs JSON schema for NN Store product copy, Roman-Urdu search synonyms, RankMath/Yoast meta tags, AEO direct answer summary, buyer FAQs, and Schema.org JSON-LD.	`class FAQItemSchema(BaseModel)` • `class GeminiSEOAEOOutputSchema(BaseModel)` • `class WooCommerceReadyPackageSchema(BaseModel)`	**Called By:** backend/app/seo_aeo/generator.py, backend/app/api/routes/seo_aeo.py **Deps:** `pydantic`
backend/app/schemas/analytics_schema.py 84 lines • 2,890 bytes	Response schemas for the ShopRaven Control Room dashboard KPIs, charts, and 30-day sales forecast.	`class DashboardKPISchema(BaseModel)` • `class CityPerformanceSchema(BaseModel)` • `class SalesForecastSchema(BaseModel)`	**Called By:** backend/app/api/routes/analytics.py **Deps:** `pydantic`

Section 7: Pakistani Market & Search Demand Scout Engine (5 Files)

Autonomous discovery layer scanning Google Trends Pakistan (gl=pk), Google Autocomplete buyer intent, Daraz.pk top-selling movers, PriceOye gadget benchmarks, and Meta Ad Library Pakistan active ads.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/scout/__init__.py 6 lines • 160 bytes	Exports PakistanMarketScout and SearchDemandScanner.	`PakistanMarketScout` • `SearchDemandScanner`	**Called By:** backend/app/tasks/pipeline_t **Deps:** `app.scout.*`
backend/app/scout/google_trends_pk.py 128 lines • 4,620 bytes	Queries Google Autocomplete PK (gl=pk, hl=en/ur) and Google Trends Pakistan rising shopping queries across NN Store's 7 categories (Beauty & Care, Kitchen, Home & Living, Gym, Electronics, Auto & Tools, Accessories).	`class GoogleTrendsPKCollector` • `fn: fetch_autocomplete_intent(category: str, seed_query: str) -> list[dict]` • `fn: fetch_rising_shopping_queries_pk() -> list[dict]`	**Called By:** backend/app/scout/demand_ **Deps:** `requests, urllib.pa app.config`
backend/app/scout/daraz_priceoye_movers.py 146 lines • 5,310 bytes	Scrapes high-velocity products and review growth velocity from Daraz.pk category leaders and PriceOye.pk electronic accessories to identify proven best-sellers in Pakistan.	`class MarketplaceVelocityScanner` • `fn: scan_daraz_category_leaders(category_slug: str) -> list[dict]` • `fn: scan_priceoye_accessories() -> list[dict]` • `fn: compute_velocity_score(reviews_count: int, rating: float) -> float`	**Called By:** backend/app/scout/demand_ **Deps:** `requests, bs4.BeautifulSoup, re`

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/scout/meta_ads_pk_verifier.py 114 lines • 4,150 bytes	Verifies whether a candidate product has active paid ads running in Pakistan (country=PK) for 7+ days with 'Cash on Delivery' or 'Shop Now' hooks, proving profitable Pakistani unit economics.	`class MetaAdLibraryVerifier` • `fn: check_active_pk_ads(keyword: str) -> dict` • `fn: score_ad_longevity(active_days: int, ad_count: int) -> float`	**Called By:** backend/app/scout/demand_ **Deps:** `requests, re, app.`

backend/app/scout/demand_scanner.py
138 lines • 4,980 bytes
	Aggregates signals from Google PK, Daraz/PriceOye, and Meta Ads into a unified composite demand score (0-100), persists new keywords to TrendKeyword table, and emits Discord MARKET_RESEARCH embeds.	`class SearchDemandScanner` • `fn: run_full_market_scan(db: Session) -> list[TrendKeyword]` • `fn: rank_candidates_by_opportunity(candidates: list[dict]) -> list[dict]`	**Called By:** backend/app/api/routes/resea
			backend/app/tasks/pipeline_t
			Deps: `app.scout.google_trends_`
			`app.scout.daraz_priceoye`
			`app.scout.meta_ads_pk_ve`

Section 8: Authenticated Dual-Supplier Vault — HHC & Oshi Dropshipping (5 Files)

Authenticated session management and product extraction for HHC Dropshipping (member.hhcdropshipping.com) and Oshi Dropshipping (portal.oshidropship.com), comparing wholesale PKR costs and stock availability.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dep
backend/app/suppliers/__init__.py 6 lines • 175 bytes	Exports HHCDropshippingClient, OshiDropshippingClient, and DualSupplierComparator.	`HHCDropshippingClient` • `OshiDropshippingClient` • `DualSupplierComparator`	**Called By:** backend/app/tas **Deps:** `app.supp`
backend/app/suppliers/session_vault.py 98 lines • 3,490 bytes	Manages persistent, encrypted session cookies in Redis so ShopRaven does not re-submit login forms on every product lookup, automatically refreshing CSRF tokens when sessions expire.	`class SupplierSessionVault` • `fn: load_cookies(supplier_key: str) -> dict` • `fn: save_cookies(supplier_key: str, cookies: dict) -> None` • `fn: extract_csrf_token(html_text: str) -> Optional[str]`	**Called By:** backend/app/sup backend/app/sup **Deps:** `redis, bs` `json`

backend/app/suppliers/hhc_client.py
154 lines • 5,620 bytes
	Authenticates into https://member.hhcdropshipping.com/en/auth/login using HHC_EMAIL and HHC_PASSWORD, searches the HHC product catalog, and extracts wholesale price (PKR), SKU, stock status, specs, and gallery URLs.	`class HHCDropshippingClient` • `fn: login() -> bool` • `fn: search_product(keyword: str) -> Optional[SupplierOfferSchema]` • `fn: fetch_product_detail(product_url: str) -> SupplierOfferSchema`	**Called By:** backend/app/sup
			Deps: `requests`
			`bs4.BeautifulS`
			`app.suppliers.`
			`app.config`

backend/app/suppliers/oshi_client.py
152 lines • 5,540 bytes
	Authenticates into https://www.portal.oshidropship.com/login using OSHI_EMAIL and OSHI_PASSWORD, queries the Oshi seller catalog, and extracts wholesale price (PKR), SKU, stock status, specs, and gallery URLs.	`class OshiDropshippingClient` • `fn: login() -> bool` • `fn: search_product(keyword: str) -> Optional[SupplierOfferSchema]` • `fn: fetch_product_detail(product_url: str) -> SupplierOfferSchema`	**Called By:** backend/app/sup
			Deps: `requests`
			`bs4.BeautifulS`
			`app.suppliers.`
			`app.config`

backend/app/suppliers/comparator.py
118 lines • 4,320 bytes
	Queries HHC and Oshi in parallel for a target product, normalizes title similarity, filters out out-of-stock offers, selects the supplier with the lowest wholesale PKR cost, saves records to supplier_offers, and notifies Discord (SUPPLIER_CHECK).	`class DualSupplierComparator` • `fn: compare_and_select_winner(keyword: str, db: Session) -> Optional[SupplierOfferSchema]` • `fn: compute_title_similarity(query: str, candidate_title: str) -> float`	**Called By:** backend/app/api
			backend/app/tas
			Deps: `app.supp`
			`app.suppliers.`
			`app.utils.disc`

Section 9: Competitive Price Researcher & +40%–50% Margin Guard Engine (4 Files)

Implements the user's exact pricing rule: +40% to +50% net profit over supplier wholesale cost with Rs. 150 separate delivery on nnstore.shop, benchmarked against live Pakistani competitor landed COD prices.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Depende
backend/app/pricing/__init__.py 5 lines • 140 bytes	Exports CompetitorPriceScraper and CompetitivePricingEngine.	`CompetitorPriceScraper` • `CompetitivePricingEngine`	**Called By:** backend/app/tasks/pi **Deps:** `app.pricing.*`
backend/app/pricing/competitor_scraper.py 124 lines • 4,480 bytes	Scrapes live Pakistani retail prices ('<product> price in pakistan Rs') across competing WooCommerce/Shopify stores and Daraz listings, filtering outliers outside [1.25x, 3.0x] of wholesale cost.	`class CompetitorPriceScraper` • `fn: fetch_pk_retail_prices(product_title: str, wholesale_cost: float) -> list[float]` • `fn: compute_trimmed_market_average(prices: list[float], fallback_cost: float) -> float`	**Called By:** backend/app/pricing/e **Deps:** `requests, re, urllib.parse`
backend/app/pricing/rounding.py 48 lines • 1,590 bytes	Psychological PKR price rounding utility matching nnstore.shop's exact catalog convention: ends every sale and regular price in *49 or *99 (e.g., Rs. 649, Rs. 749, Rs. 899, Rs. 1,099, Rs. 1,399).	`fn: round_to_nnstore_ending(raw_price: float, min_floor: float) -> float` • `fn: compute_strikethrough_regular_price(sale_price: float) -> float`	**Called By:** backend/app/pricing/e **Deps:** `math`
backend/app/pricing/engine.py 142 lines • 5,190 bytes	Core pricing decision engine. Compares Competitor Landed Cost (Market Avg + Rs. 220 avg courier) against NN Store Landed Cost (Sale Price + Rs. 150 separate delivery), positioning the sale price inside [Cost * 1.40, Cost * 1.50] to maximize profit while undercutting competitors at the doorstep.	`class CompetitivePricingEngine` • `fn: calculate_optimal_price(offer: SupplierOfferSchema, db: Session) -> CompetitivePricingResponseSchema`	**Called By:** backend/app/api/route backend/app/tasks/pi **Deps:** `app.pricing.compet` `app.pricing.roundi` `app.models.pricing`

Section 10: Automated 1080×1080 Product Image Studio & AI Background Engine (4 Files)

Downloads raw product photos from HHC and Oshi, filters out low-res icons or watermarked frames, centers products onto a 1080×1080 studio canvas, enhances clarity/contrast, and uploads media to WordPress.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/studio/__init__.py 5 lines • 135 bytes	Exports ProductImageStudio and WordPressMediaUploader.	`ProductImageStudio` • `WordPressMediaUploader`	**Called By:** backend/app/tasks/pipeline_task **Deps:** `app.studio.*`
backend/app/studio/image_processor.py 136 lines • 4,890 bytes	Downloads raw supplier images, validates minimum dimensions (>=400×400), resizes proportionally with LANCZOS resampling onto a 1080×1080 soft-studio canvas (#FAFAFC), and applies +6% contrast and +12% sharpness.	`class ProductImageStudio` • `fn: download_and_enhance_gallery(image_urls: list[str], slug: str) -> list[str]` • `fn: standardize_to_square_canvas(img: Image.Image, size: int = 1080) -> Image.Image`	**Called By:** backend/app/tasks/pipeline_task **Deps:** `PIL.Image, PIL.ImageEnhance, requests, io.BytesIO`
backend/app/studio/badge_overlay.py 84 lines • 2,910 bytes	Implements Alippo Creative Studio feature: optionally renders clean promotional offer banners or feature callout badges onto secondary gallery thumbnails.	`class StudioBadgeRenderer` • `fn: apply_feature_callout(image_path: str, headline: str) -> str`	**Called By:** backend/app/studio/image_proce **Deps:** `PIL.Image, PIL.ImageD` `PIL.ImageFont`

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/studio/wp_media_uploader.py 92 lines • 3,280 bytes	Uploads processed 1080×1080 JPEG files directly to https://nnstore.shop/wp-json/wp/v2/media and attaches localized Pakistani SEO alt_text and title attributes.	`class WordPressMediaUploader` • `fn: upload_image_to_wp(local_path: str, alt_text: str, title: str) -> dict`	**Called By:** backend/app/tasks/pipeline_tasks **Deps:** `requests, base64, app.config`

Section 11: SEO, AEO (Answer Engine Optimization) & JSON-LD Schema Engine (4 Files)

Generates NN Store's exact short description format (1-line bold hook + 4 bullets), full persuasive HTML copy, RankMath & Yoast SEO meta tags, AEO direct-answer summaries, and valid Schema.org Product + FAQPage JSON-LD.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/seo_aeo/__init__.py 5 lines • 140 bytes	Exports NNStoreSEOAEOEngine and JSONLDSchemaBuilder.	`NNStoreSEOAEOEngine` • `JSONLDSchemaBuilder`	**Called By:** backend/app/tasks/pipeline_ta **Deps:** `app.seo_aeo.*`
backend/app/seo_aeo/prompts.py 108 lines • 4,120 bytes	System and user prompt templates for Gemini 2.5 Flash instructing it to write in nnstore.shop's brand voice, incorporate flat Rs. 150 COD shipping and WhatsApp +92 312 4999196, and target Pakistani buyer queries.	`CONST: NNSTORE_SEO_AEO_SYSTEM_PROMPT` • `fn: build_product_seo_prompt(offer: dict, pricing: dict, category: str) -> str`	**Called By:** backend/app/seo_aeo/generat **Deps:** `json`
backend/app/seo_aeo/json_ld_builder.py 116 lines • 4,290 bytes	Constructs deterministic, W3C-valid Schema.org @graph containing Product, Offer (priceCurrency: PKR), OfferShippingDetails (MonetaryAmount: 150 PKR, addressCountry: PK), and FAQPage.	`class JSONLDSchemaBuilder` • `fn: build_product_and_faq_schema(slug: str, title: str, sku: str, sale_price: float, images: list[str], aeo_summary: str, faqs: list[dict]) -> dict`	**Called By:** backend/app/seo_aeo/generat **Deps:** `json`
backend/app/seo_aeo/generator.py 158 lines • 5,840 bytes	Invokes Gemini 2.5 Flash with responseMimeType='application/json' (plus deterministic fallback), assembles short_description_html, full_description_html with embedded JSON-LD script, and RankMath meta keys.	`class NNStoreSEOAEOEngine` • `fn: generate_package(offer: SupplierOfferSchema, pricing: CompetitivePricingResponseSchema, category: str) -> WooCommerceReadyPackageSchema`	**Called By:** backend/app/api/routes/seo_a backend/app/tasks/pipeline_ta **Deps:** `requests, app.seo_aeo.prompts, app.seo_aeo.json_ld_build app.seo_aeo.roman_urdu_ma`

Section 12: Roman-Urdu & Misspelling Search Synonym Mapper — Alippo Feature #4 (3 Files)

Solves the '0 search results' problem demonstrated in Alippo's Search Optimization Reel by mapping local Pakistani Roman-Urdu phrases and phonetic typos to every product's search metadata.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Depe
backend/app/seo_aeo/roman_urdu_dictionary.json 140 lines • 5,120 bytes	Curated lexicon mapping NN Store's 7 product categories to common Pakistani Roman-Urdu terms and phonetic misspellings (e.g., 'daane aur blackhead nikalne wali machine', 'kaano ki tooti / earbuds', 'kapron se bur utarne wali machine').	`JSON mapping by category and product archetype`	**Called By:** backend/app/seo_ **Deps:** `Standard J`
backend/app/seo_aeo/roman_urdu_mapper.py 96 lines • 3,480 bytes	Generates 15-20 localized Roman-Urdu synonyms, phonetic spelling variations, and regional terms per product, injecting them into WooCommerce tags and custom search meta_data (_shopraven_search_synonyms).	`class RomanUrduSearchMapper` • `fn: generate_local_synonyms(product_title: str, category: str) -> list[str]` • `fn: generate_phonetic_typos(keyword: str) -> list[str]`	**Called By:** backend/app/seo_ **Deps:** `json, app.`
backend/tests/test_roman_urdu_mapper.py 54 lines • 1,840 bytes	Unit tests verifying that Roman-Urdu queries map to the right NN Store products with zero empty results.	`fn: test_skin_scrubber_roman_urdu_synonyms()` • `fn: test_typo_generation()`	**Called By:** pytest s **Deps:** `pytest, app.seo_aeo.rom`

Section 13: Deal Hunter Price-Hesitation Coupon & Margin-Floor Bot — Alippo Feature #5 (3 Files)

Replicates Alippo's 'Deal Hunter' reel: detects price-sensitive shoppers who hesitate on product or cart pages and generates a one-time coupon that converts the sale without ever violating your +40% minimum profit floor.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/coupons/__init__.py 4 lines • 110 bytes	Exports DealHunterCouponEngine.	`DealHunterCouponEngine`	**Called By:** backend/app/api/route **Deps:** `app.coupons.deal_hunte`
backend/app/coupons/deal_hunter.py 122 lines • 4,490 bytes	Calculates the maximum safe discount (PKR or %) for a given cart based on each item's wholesale cost so that (Sale Price - Discount) >= (Wholesale Cost * 1.40), then creates a 30-minute single-use coupon code.	`class DealHunterCouponEngine` • `fn: compute_margin_safe_discount(sale_price_pkr: float, wholesale_cost_pkr: float) -> dict` • `fn: generate_one_time_coupon_payload(code_prefix: str, discount_pkr: float) -> dict`	**Called By:** orchestrator/src/woocommerce/c backend/app/api/routes/pricing.p **Deps:** `secrets, datetime, ap`
backend/tests/test_deal_hunter.py 62 lines • 2,120 bytes	Verifies that generated Deal Hunter coupons never reduce net profit below +40% of HHC/Oshi wholesale cost.	`fn: test_coupon_respects_40pct_floor()` • `fn: test_zero_discount_when_at_floor()`	**Called By:** pytest suite **Deps:** `pytest, app.coupons.de`

Section 14: Autonomous SEO Blog Generator with Embedded Product Cards — Alippo Feature #7 (3 Files)

Replicates Alippo's SEO Blog Reel: writes high-ranking Pakistani buying guides around trending queries and embeds clickable WooCommerce product cards ([products ids='...']) directly inside the article.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/blog/__init__.py 4 lines • 115 bytes	Exports SEOBlogPublisher.	`SEOBlogPublisher`	**Called By:** backend/app/tasks/pipeline_tasks **Deps:** `app.blog.auto_blogger`
backend/app/blog/auto_blogger.py 148 lines • 5,420 bytes	Generates 900-word SEO articles targeting Pakistani informational queries (e.g., 'Best Skincare & Pore Cleansing Tools in Pakistan Under Rs. 2,500') and injects inline WooCommerce [products ids='...'] shortcodes and custom HTML product cards with 1-click 'Add to Cart' links.	`class SEOBlogPublisher` • `fn: generate_blog_post_with_product_cards(topic: str, wc_products: list[dict]) -> dict` • `fn: publish_to_wordpress(post_payload: dict) -> dict`	**Called By:** backend/app/api/routes/blog.py, backend/app/tasks/pipeline_tasks **Deps:** `requests, app.config, app.utils.discord_notifier`
backend/app/blog/product_card_template.py 74 lines • 2,680 bytes	Renders responsive inline HTML product cards with Sale badge, PKR price, and direct '/?add-to-cart=ID' CTA.	`fn: render_inline_product_card_html(product: dict) -> str`	**Called By:** backend/app/blog/auto_blogger.p **Deps:** `html`

Section 15: 30-Day AI Sales Forecaster & True Net Profit Accounting — Alippo Features #2 & #13 (4 Files)

Calculates real net take-home profit in PKR (after HHC/Oshi product cost, courier charges, packaging, and COD RTO losses), predicts 30-day SKU demand, and generates automated monthly executive reports.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Depende
backend/app/forecasting/__init__.py 5 lines • 145 bytes	Exports TrueProfitAccountant, SalesForecaster, and MonthlyReportBuilder.	`TrueProfitAccountant` • `SalesForecaster` • `MonthlyReportBuilder`	**Called By:** backend/ap **Deps:** `app.forecasti`
backend/app/forecasting/true_profit_accountant.py 118 lines • 4,210 bytes	Computes exact realized net profit per order and per month: Net Profit = Delivered Revenue + Shipping Collected - (HHC/Oshi Cost + Courier Dispatch Fee + Packaging + RTO Return Penalty).	`class TrueProfitAccountant` • `fn: compute_order_net_profit(order: dict) -> dict` • `fn: compute_period_pnl_statement(orders: list[dict]) -> dict`	**Called By:** backend/ap backend/app/forecasti **Deps:** `app.models.an`
backend/app/forecasting/sales_predictor.py 132 lines • 4,760 bytes	Uses exponential smoothing and search-trend velocity to forecast next-30-day order volume and PKR revenue per SKU, alerting you on Discord before a top seller runs out of stock at HHC or Oshi.	`class SalesForecaster` • `fn: forecast_next_30_days(snapshots: list[AnalyticsSnapshot]) -> dict` • `fn: predict_sku_restock_urgency(wc_product_id: int, daily_velocity: float) -> dict`	**Called By:** backend/ap **Deps:** `math, statist` `app.models.analyti`
backend/app/forecasting/monthly_report.py 104 lines • 3,820 bytes	Replicates Alippo's Monthly Report Reel: compiles a complete monthly executive digest (Revenue, True Net Profit, Best Sellers, RTO Rate, Top Cities, and AI Recommendations) and posts it to Discord.	`class MonthlyReportBuilder` • `fn: generate_monthly_executive_digest(db: Session) -> dict`	**Called By:** backend/ap **Deps:** `app.forecasting.tr` `app.utils.discord_`

Section 16: Python FastAPI Routers, Background Tasks & Discord Utility (8 Files)

REST API controllers and Celery background job runners exposing all Python intelligence modules to the Node.js Orchestrator and Control Room Frontend.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/api/router.py 42 lines • 1,380 bytes	Aggregates all v1 API sub-routers under /api/v1.	`api_router = APIRouter(prefix='/api/v1')`	**Called By:** backend/app/main.py **Deps:** `fastapi, app.api.routes.*`
backend/app/api/routes/research.py 74 lines • 2,580 bytes	Endpoints to trigger Pakistani market scans and list trending keywords with catalog gap status.	`POST /api/v1/research/scan` • `GET /api/v1/research/trends`	**Called By:** orchestrator/src/jobs/cron_schec frontend Control Room **Deps:** `fastapi, app.scout.demand_scan`
backend/app/api/routes/suppliers.py 78 lines • 2,710 bytes	Endpoints to compare HHC Dropshipping vs Oshi Dropshipping for any keyword or SKU.	`POST /api/v1/suppliers/compare` • `GET /api/v1/suppliers/scorecard`	**Called By:** orchestrator/src/jobs/cron_schec frontend Control Room **Deps:** `fastapi, app.suppliers.compara`

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
backend/app/api/routes/pricing.py 82 lines • 2,890 bytes	Endpoints for competitive price calculation (+40%-50% + Rs. 150 COD) and Deal Hunter coupon generation.	`POST /api/v1/pricing/calculate` • `POST /api/v1/pricing/deal-hunter-coupon`	**Called By:** orchestrator/src/woocommerce/product_pu frontend Simulator **Deps:** `fastapi, app.pricing.engine, app.coupons.deal_hunter`
backend/app/api/routes/seo_aeo.py 76 lines • 2,690 bytes	Endpoints to generate WooCommerce SEO/AEO copy, Roman-Urdu synonyms, and JSON-LD schema.	`POST /api/v1/seo-aeo/generate` • `POST /api/v1/seo-aeo/audit-store`	**Called By:** orchestrator/src/woocommerce/product_pu **Deps:** `fastapi, app.seo_aeo.generator`
backend/app/api/routes/blog.py 64 lines • 2,190 bytes	Endpoints to generate and publish SEO blog articles with embedded WooCommerce product cards.	`POST /api/v1/blog/generate-and-publish`	**Called By:** orchestrator/src/jobs/cron_schec **Deps:** `fastapi, app.blog.auto_blogger`
backend/app/api/routes/analytics.py 96 lines • 3,420 bytes	Serves unified executive analytics, traffic, new vs. returning visitors, city COD breakdown, and 30-day forecast.	`GET /api/v1/analytics/dashboard` • `GET /api/v1/analytics/forecast`	**Called By:** frontend Control Room, orchestrator/src/server.js **Deps:** `fastapi, app.forecasting.*`
backend/app/tasks/pipeline_tasks.py 164 lines • 6,120 bytes	End-to-end autonomous pipeline task chaining Step 1 (Market Scout) → Step 2 (Catalog Gap Check) → Step 3 (HHC vs Oshi Lookup) → Step 4 (Competitive Price +40-50%) → Step 5 (Studio Images) → Step 6 (SEO/AEO + JSON-LD) → Step 7 (WooCommerce Draft + Discord Alert).	`task: execute_autonomous_product_pipeline(keyword: str, category: str)` • `task: sync_supplier_stock_and_waitlists()`	**Called By:** backend/app/celery_app.py, orchestrator/src/jobs/cron_scheduler.js **Deps:** `app.scout, app.suppliers, app.pricing, app.studio, app.seo_ae`

Section 17: Node.js Orchestrator Core, Cron Scheduler & WooCommerce Client (8 Files)

Node.js orchestration service (orchestrator/) managing real-time WordPress/WooCommerce synchronization for https://nnstore.shop, scheduled automation loops, and multi-channel webhooks.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols
orchestrator/package.json 38 lines • 1,120 bytes	Node.js package manifest for the ShopRaven Orchestrator and Discord Bot gateway.	`name: shopraven-orchestrator` • `scripts: start, dev, test`
orchestrator/Dockerfile 28 lines • 740 bytes	Lightweight Node.js 20-alpine container image for shopraven-orchestrator.	`FROM node:20-alpine` • `CMD node src/server.js`

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols
orchestrator/src/config/env.js 64 lines • 2,180 bytes	Loads .env variables and exports typed configuration for nnstore.shop, WooCommerce REST API, and Discord.	`CONFIG` • `NNSTORE_CATEGORIES`
orchestrator/src/server.js 142 lines • 5,120 bytes	HTTP/REST server listening on port 4000: receives step progress webhooks from the Python engine (/api/notify), WooCommerce order/checkout webhooks (/webhooks/woocommerce), and serves the Control Room.	`POST /api/notify` • `POST /webhooks/woocommerce/order-created` • `POST /webhooks/woocommerce/cart-abandoned` • `POST /api/publish-draft/:id`
orchestrator/src/jobs/cron_scheduler.js 108 lines • 3,890 bytes	Runs scheduled background jobs: (1) Every 6h: Market & Search Demand Scan; (2) Every 2h: HHC & Oshi Stock Sync & Restock Waitlist Trigger; (3) Every Sunday 02:00 PKT: Store Menu Reordering & Weekly SEO Blog.	`fn: startScheduledJobs()` • `cron: '0 */6 * * *' (Market Research Loop)` • `cron: '0 */2 * * *' (Supplier Stock & Waitlist Sync)` • `cron: '0 2 * * 0' (Menu Reorder & Auto-Blogger)`
orchestrator/src/woocommerce/wc_api_client.js 116 lines • 4,120 bytes	Authenticated HTTP client for https://nnstore.shop/wp-json/wc/v3/ supporting GET, POST, PUT, and DELETE with Basic Auth (WC_CONSUMER_KEY:WC_CONSUMER_SECRET) and public Store API v1 fallback.	`class WooCommerceAPIClient` • `fn: get(endpoint, params)` • `fn: post(endpoint, body)` • `fn: put(endpoint, body)`
orchestrator/src/woocommerce/catalog_sync.js 134 lines • 4,780 bytes	Fetches all live and draft products from nnstore.shop, performs token-similarity Catalog Gap Checks ('Does my store have this product or not?'), and flags missing opportunities on Discord.	`class CatalogGapChecker` • `fn: fetchAllStoreProducts()` • `fn: checkProductExistsInStore(trendingKeyword)`
orchestrator/src/woocommerce/product_publisher.js 148 lines • 5,340 bytes	Creates Draft WooCommerce products on nnstore.shop with Regular & Sale prices, 1-line hook + 4 bullets short description, full description + JSON-LD schema, RankMath/Yoast meta_data, Roman-Urdu search tags, and private HHC/Oshi cost tracking fields, plus 1-click transition from Draft to Publish.	`class WooCommerceProductPublisher` • `fn: createDraftListing(agentPackage)` • `fn: publishDraftToLive(wcProductId)`

Section 18: Storefront Automation: Menu Reorder, 301 Redirects & Restock Waitlist — Alippo Features #6, #8, #9 (4 Files)

Automates storefront merchandising on nnstore.shop: reorders menus by sales velocity, redirects dead/removed product URLs to active alternatives, and sends 1-tap WhatsApp restock links.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Depend
orchestrator/src/woocommerce/menu_reorder.js 112 lines • 3,980 bytes	Replicates Alippo Feature #6 (Auto-Reorder Store Menu by Sales): computes conversion velocity (Orders / Views) for all products on nnstore.shop and updates WooCommerce menu_order and featured flags so best-sellers appear first on category pages and homepage shelves.	`class StorefrontMenuOptimizer` • `fn: reorderCatalogByVelocity()` • `fn: rankCategoryPerformance()`	**Called By:** orchestra **Deps:** orchestrator/src/ orchestrator/src/
orchestrator/src/woocommerce/smart_redirects.js 104 lines • 3,740 bytes	Replicates Alippo Feature #9 (Dead-Product Smart Auto-Redirect): when an item is discontinued by both HHC and Oshi, finds the closest in-stock product in the same category and registers a 301 redirect on WordPress so ad traffic and SEO links never hit a 404 error.	`class SmartRedirectManager` • `fn: findClosestActiveReplacement(removedProduct, activeCatalog)` • `fn: apply301Redirect(oldSlug, targetProductUrl)`	**Called By:** orchestra **Deps:** orchestrator/src/

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Depend
`orchestrator/src/woocommerce/restock_notifier.js` 118 lines • 4,210 bytes	Replicates Alippo Feature #8 (Out-of-Stock 'Notify Me on WhatsApp'): monitors HHC and Oshi stock every 2 hours; when a backordered SKU returns to stock, marks it instock on nnstore.shop and sends every waitlisted buyer a 1-tap WhatsApp checkout link (https://nnstore.shop/?add-to-cart=ID).	`class RestockWaitlistDispatcher` • `fn: registerWaitlistSubscriber(wcProductId, whatsappNumber)` • `fn: dispatchRestockLinksForProduct(wcProductId)`	**Called By:** orchestra orchestrator/src/serv **Deps:** `orchestrator/src/` `orchestrator/src/`
`orchestrator/src/woocommerce/coupon_service.js` 86 lines • 3,040 bytes	Creates single-use WooCommerce coupons via /wp-json/wc/v3/coupons for the Deal Hunter bot and Prepaid incentives.	`class WooCommerceCouponService` • `fn: createSingleUseCoupon(code, amountPKR, expiryMinutes)`	**Called By:** orchestra orchestrator/src/wha **Deps:** `orchestrator/src/`

Section 19: COD-to-Prepaid Split, RTO Risk Scorer & WhatsApp Cart Recovery — Alippo Features #11 & #12 (4 Files)

Protects nnstore.shop from Pakistani Cash-on-Delivery return losses (RTO) and recovers abandoned checkouts via automated WhatsApp messages (+92 312 4999196).

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
`orchestrator/src/whatsapp/cloud_client.js` 92 lines • 3,290 bytes	Wrapper for Meta WhatsApp Cloud API dispatching transactional templates and 1-tap cart links.	`class WhatsAppCloudClient` • `fn: sendTextMessage(recipientPhonePK, bodyText)`	**Called By:** cart_recovery.js, cod restock_notifier.js **Deps:** `orchestrator/src/con`
`orchestrator/src/whatsapp/cart_recovery.js` 126 lines • 4,540 bytes	Replicates Alippo Feature #12 (WhatsApp Abandoned-Cart Recovery): triggers 20 minutes after checkout abandonment, sending a personalized WhatsApp message with a pre-filled checkout link and optional Deal Hunter coupon.	`class AbandonedCartRecoveryEngine` • `fn: handleAbandonedCheckout(checkoutPayload)` • `fn: buildPreFilledRecoveryUrl(cartItems, couponCode)`	**Called By:** orchestrator/src/serv **Deps:** `orchestrator/src/whatsapp/` `orchestrator/src/woocomme`

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
orchestrator/src/whatsapp/cod_rto_guard.js 138 lines • 4,960 bytes	Replicates Alippo Feature #11 (COD-to-Prepaid Split & RTO Protection): scores incoming COD orders for address completeness and past return history, sends an instant WhatsApp order confirmation request, and offers to waive the Rs. 150 delivery fee if the buyer switches to prepaid (Raast / EasyPaisa / JazzCash).	`class CODRTOGuard` • `fn: scoreOrderRTORisk(orderPayload) -> { riskScore, riskLevel, reasons }` • `fn: sendCODConfirmationAndPrepaidOffer(orderPayload)`	**Called By:** orchestrator/src/serv **Deps:** `orchestrator/src/whatsapp/` `orchestrator/src/discord/`
orchestrator/tests/orchestrator.test.js 84 lines • 2,950 bytes	Automated test suite verifying catalog gap matching, RTO risk scoring, and Discord payload formatting.	`test: checkCatalogGap()` • `test: scoreOrderRTORisk()`	**Called By:** npm test / CI pipeline **Deps:** `jest`

Section 20: Discord Rich Embed Notifier & 1-Click Publish Bot Commands (3 Files)

Real-time Discord command center integration broadcasting color-coded Rich Embeds for every step of the AI pipeline and allowing 1-click product publishing directly from Discord.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
orchestrator/src/discord/notifier.js 114 lines • 4,080 bytes	Formats and dispatches color-coded Discord Rich Embeds for MARKET_RESEARCH, CATALOG_GAP_CHECK, SUPPLIER_CHECK, PRICE_RESEARCH, IMAGE_STUDIO, SEO_AEO_ENGINE, WOOCOMMERCE_DRAFT, and COD_RTO_GUARD.	`class DiscordNotifier` • `fn: sendEmbed(step, status, message, fieldsObj)`	**Called By:** All orchestrator modules & webhook **Deps:** `orchestrator/src/config/e`
orchestrator/src/discord/bot_commands.js 132 lines • 4,720 bytes	Interactive Discord slash commands (/scan-now, /publish <draft_id>, /price-check <keyword>, /pnl-report, /reorder-menu) so you can control ShopRaven from your phone on Discord.	`class ShopRavenDiscordBot` • `command: /scan-now` • `command: /publish <wc_product_id>` • `command: /price-check <keyword>` • `command: /pnl-report`	**Called By:** orchestrator/src/server.js **Deps:** `orchestrator/src/woocommerce/p`

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
orchestrator/src/discord/embed_templates.js 88 lines • 3,120 bytes	Standardized Discord embed templates with direct WP Admin edit links and HHC/Oshi supplier links.	fn: buildDraftCreatedEmbed(draftData) • fn: buildMonthlyReportEmbed(pnlData)	**Called By:** orchestrator/src/discord/n **Deps:** None

Section 21: WordPress / WooCommerce Plugin for nnstore.shop — Alippo Features #8, #10, #11, #13 (5 Files)

Lightweight WordPress plugin (wordpress-plugin/shopraven-bridge/) installed on https://nnstore.shop to render the Converting Cart Progress Bar, Top-Bar Prepaid Discount Banner, Out-of-Stock WhatsApp Box, and 24/7 Product Q&A Bot.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
wordpress-plugin/shopraven-bridge/shopraven-bridge.php 168 lines • 6,240 bytes	Main WordPress plugin file registering WooCommerce hooks for the Cart Progress Bar, Prepaid Rs. 150 Shipping Waiver, Roman-Urdu search query expansion, and Out-of-Stock WhatsApp waitlist widget.	action: woocommerce_before_cart_contents (Converting Cart Progress Bar) • filter: pre_get_posts (Roman-Urdu Search Synonym Expansion) • action: woocommerce_single_product_summary (Out-of-Stock WhatsApp Capture) • action: wp_footer (24/7 AI Product Assistant Widget)	**Called By:** WordPress Core on https://nnstore.shop **Deps:** WooCommerce 8+
wordpress-plugin/shopraven-bridge/includes/class-cart-progress-bar.php 94 lines • 3,380 bytes	Replicates Alippo Feature #10 (Converting Cart Progress Bar): displays a dynamic progress bar inside the cart drawer ('Add Rs. X more to unlock FREE Delivery — Save Rs. 150!') plus 1-click AI bundle upsells.	class ShopRaven_Cart_Progress_Bar • fn: render_progress_bar() • fn: apply_free_shipping_above_threshold($rates, $package)	**Called By:** shopraven-bridge.php **Deps:** WooCommerce Cart API
wordpress-plugin/shopraven-bridge/includes/class-prepaid-incentive.php 88 lines • 3,150 bytes	Replicates Alippo Feature #11 (Top-Bar & Checkout Prepaid Discount): displays a top-bar banner and checkout toggle waiving the flat Rs. 150 shipping fee (or applying 5% off) when paying via Raast/Bank Transfer.	class ShopRaven_Prepaid_Incentive • fn: render_top_bar_banner() • fn: apply_prepaid_shipping_waiver(WC_Cart $cart)	**Called By:** shopraven-bridge.php **Deps:** WooCommerce Checkout API
wordpress-plugin/shopraven-bridge/includes/class-product-qa-bot.php 116 lines • 4,190 bytes	Replicates Alippo Feature #13 (24/7 On-Page Product Q&A Bot): embeds a floating product-page AI assistant pre-loaded with the product's specs, PKR price, 3-5 day COD delivery timeline, and WhatsApp escalation.	class ShopRaven_Product_QA_Bot • fn: render_product_assistant_widget() • fn: register_rest_qa_endpoint()	**Called By:** shopraven-bridge.php **Deps:** WordPress REST API
wordpress-plugin/shopraven-bridge/assets/shopraven-storefront.js 142 lines • 4,980 bytes	Client-side storefront script handling price-hesitation dwell detection (Deal Hunter popup trigger), WhatsApp waitlist form submission, and real-time visitor telemetry.	fn: initPriceHesitationDetector() • fn: submitWhatsAppRestockWaitlist(productId, phone) • fn: askProductQABot(productId, question)	**Called By:** shopraven-bridge.php **Deps:** Vanilla ES6 JavaScript

Section 22: ShopRaven Control Room Frontend & Analytics Dashboard (8 Files)

Interactive Control Room web application providing real-time visibility into Total Traffic, New vs. Returning Visitors, Total Sales Revenue (PKR), Net Realized Profit (+40-50%), COD Order Lifecycle, City Heatmap, Trending Keywords, HHC vs. Oshi Scorecard, Interactive Price Simulator, and WooCommerce Drafts Queue.

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
frontend/nnstore_ai_dashboard.html 865 lines • 38,940 bytes	Self-contained single-file production Control Room dashboard with 5 interactive views: Executive Analytics, Trending Keywords & Gap Finder, HHC vs Oshi & Price Simulator, SEO/AEO & Drafts Queue, and Live Discord Feed.	fn: initCharts() • fn: renderTimeframe(rangeKey) • fn: renderCityBreakdown() • fn: renderKeywordsTable() • fn: runPricingSimulator() • fn: triggerLiveAICycle()	**Called By:** Browser / Canvas / orchestrator static server **Deps:** Tailwind CSS 3.4, Chart.js 4.4

File Path & Metrics	Primary Purpose & Antigravity Contract	Declared Classes, Functions & Symbols	Call Graph & Dependencies
`frontend/src/App.tsx` 148 lines • 5,210 bytes	Modular React 18 + TypeScript entrypoint for the multi-page ShopRaven Control Room SPA.	`export default function App()`	**Called By:** frontend/src/main.tsx **Deps:** `react, zustand, lucide-react`
`frontend/src/store/useShopRavenStore.ts` 112 lines • 3,940 bytes	Global Zustand state store managing selected timeframe, live KPI metrics, keyword queue, and Discord logs.	`useShopRavenStore` • `action: setTimeframe` • `action: triggerResearchCycle` • `action: publishDraft`	**Called By:** All frontend/src/components/*.tsx views **Deps:** `zustand`
`frontend/src/components/ExecutiveAnalyticsView.tsx` 184 lines • 6,490 bytes	Renders the 8 Primary & Operational KPI cards, Daily Sales vs Net Profit chart, New vs Returning Visitors doughnut chart, Pakistani City COD progress bars, Traffic Channels polar chart, and Top Selling Products.	`export function ExecutiveAnalyticsView()`	**Called By:** frontend/src/App.tsx **Deps:** `react, chart.js, useShopRavenStore`
`frontend/src/components/KeywordGapFinderView.tsx` 142 lines • 5,120 bytes	Interactive table of trending Pakistani search keywords with instant 'Auto-Import Draft' and 'Refresh SEO/AEO' actions.	`export function KeywordGapFinderView()`	**Called By:** frontend/src/App.tsx **Deps:** `react, useShopRavenStore`
`frontend/src/components/SupplierPriceSimulatorView.tsx` 164 lines • 5,890 bytes	Displays the HHC vs Oshi Supplier Scorecard and interactive +40%-50% Competitive Landed COD Price Simulator.	`export function SupplierPriceSimulatorView()`	**Called By:** frontend/src/App.tsx **Deps:** `react, useShopRavenStore`
`frontend/src/components/SEOAEOQueueView.tsx` 156 lines • 5,540 bytes	Manages the WooCommerce Drafts Queue, JSON-LD Schema Inspector Modal, and 1-Click Publish to nnstore.shop.	`export function SEOAEOQueueView()`	**Called By:** frontend/src/App.tsx **Deps:** `react, useShopRavenStore`
`frontend/src/components/DiscordLiveFeedView.tsx` 98 lines • 3,410 bytes	Streams real-time Discord Rich Embed logs and allows triggering test webhook alerts.	`export function DiscordLiveFeedView()`	**Called By:** frontend/src/App.tsx **Deps:** `react, useShopRavenStore`

Section 23: Complete Alippo 13-Feature Parity Matrix & Mathematical Formulas

Every capability showcased across the Alippo AI Cofounder website, brochure, and reels is mapped to its corresponding ShopRaven module below:

#	Alippo Feature	ShopRaven Implementation Module	Exact Formula / Technical Mechanism
01	Bulk Photo-to-Draft Listing & Auto-Collection	`ProductImageStudio` + `WooCommerceProductPublisher`	Standardizes gallery to 1080×1080 JPEG and maps to `nnstore.shop`'s 7 WooCommerce categories.
02	True Net Profit & Hidden Cost Accounting	`TrueProfitAccountant` (`true_profit_accountant.py`)	`Net_Profit = Sale_Price - Wholesale_Cost - Packaging - (RTO_Rate * Courier_Loss)`
03	Built-In AI SEO & AEO Schema	`NNStoreSEOAEOEngine` + `JSONLDSchemaBuilder`	Populates RankMath/Yoast fields + `Product`, `OfferShippingDetails (Rs 150)`, & `FAQPage` JSON-LD.
04	Regional Language & Typo Search Mapping	`RomanUrduSearchMapper` (`roman_urdu_mapper.py`)	Injects 15–20 Roman-Urdu phrases & phonetic typos into `_shopraven_search_synonyms`.
05	Deal Hunter Price-Hesitation Coupon Bot	`DealHunterCouponEngine` (`deal_hunter.py`)	`Max_Discount = max(0, Sale_Price - (Wholesale_Cost * 1.40))` via `/wp-json/wc/v3/coupons`.

#	Alippo Feature	ShopRaven Implementation Module	Exact Formula / Technical Mechanism
06	Auto-Reorder Store Menu by Sales Velocity	`StorefrontMenuOptimizer` (`menu_reorder.js`)	`Velocity = (Orders * 3.0) + (Views * 0.05);` updates WooCommerce `menu_order` weekly.
07	SEO Auto-Blogger with Embedded Product Cards	`SEOBlogPublisher` (`auto_blogger.py`)	Publishes buying guides to `/wp-json/wp/v2/posts` with inline `/?add-to-cart=ID` cards.
08	Out-of-Stock "Notify Me on WhatsApp" 1-Tap Link	`RestockWaitlistDispatcher` (`restock_notifier.js`)	Watches HHC & Oshi stock every 2h; texts waitlisted buyers a 1-tap checkout link on restock.
09	Dead-Product Smart 301 Auto-Redirect	`SmartRedirectManager` (`smart_redirects.js`)	Redirects discontinued product slugs to highest-converting active SKU in the same category.
10	Converting Cart Progress Bar & AI Bundles	`ShopRaven_Cart_Progress_Bar` (WP Plugin)	Shows dynamic progress bar in cart unlocking free Rs. 150 shipping above `Rs. 2,500` cart total.
11	COD-to-Prepaid Split & RTO Risk Scorer	`CODRTOGuard` (`cod_rto_guard.js`)	Scores COD orders for RTO risk; waives Rs. 150 delivery fee when buyer switches to Raast/EasyPaisa.
12	WhatsApp Abandoned Cart Recovery & Segments	`AbandonedCartRecoveryEngine` (`cart_recovery.js`)	Sends pre-filled cart URL 20 mins after abandonment; tags buyers as VIP, Repeat, or Deal Hunter.
13	24/7 Product Q&A Bot & 30-Day Sales Forecaster	`ShopRaven_Product_QA_Bot` + `SalesForecaster`	Answers buyer questions on product pages 24/7 and forecasts 30-day SKU demand for restock alerts.

Section 24: Antigravity IDE Master Injection Prompt & Build Execution Sequence

Copy and paste the prompt block below into **Antigravity** along with this PDF attached to scaffold, wire, and test the entire ShopRaven repository in deterministic stages:

```
SYSTEM INSTRUCTION FOR ANTIGRAVITY IDE:
You are building the "ShopRaven" Autonomous E-Commerce AI Cofounder platform (GitHub: NaeeemJatt/ShopRaven) for https://nnstore.shop
(WordPress / WooCommerce).
Follow the attached "ShopRaven — Comprehensive System Architecture & Codebase Manual" PDF strictly.

1. LOCKED BUSINESS & ARCHITECTURE RULES:
   - Storefront: https://nnstore.shop (7 WooCommerce categories: Beauty & Care, Kitchen Accessories, Home & Living, Gym Accessories,
Electronic Accessories, Auto & Tools, Accessories).
   - Suppliers: HHC Dropshipping (https://member.hhcdropshipping.com/en/auth/login) and Oshi Dropshipping
(https://www.portal.oshidropship.com/login) via authenticated session scraping.
   - Pricing Engine: +40% to +50% profit margin over winning supplier wholesale cost (PKR), with flat Rs. 150 delivery billed separately
at checkout, benchmarked against live Pakistani competitor landed COD prices, rounded to *49 or *99 endings.
   - Notifications: Discord Rich Embeds at every pipeline step + interactive slash commands.

2. STEP-BY-STEP BUILD SEQUENCE:
   - Sprint 1: Scaffold root config (.env.example, docker-compose.yml, Makefile) and Python backend core (backend/app/config.py,
database.py, models/, schemas/, alembic migrations 001-007).
   - Sprint 2: Implement PakistanMarketScout (google_trends_pk.py, daraz_priceoye_movers.py, meta_ads_pk_verifier.py, demand_scanner.py)
and Authenticated Supplier Vault (session_vault.py, hhc_client.py, oshi_client.py, comparator.py).
   - Sprint 3: Implement CompetitivePricingEngine (+40-50% margin + Rs. 150 COD landed math), ProductImageStudio (1080x1080 square
canvas + WP media upload), and NNStoreSEOAEOEngine (Roman-Urdu mapper, RankMath meta, JSON-LD Product + OfferShippingDetails + FAQPage).
   - Sprint 4: Implement Alippo Parity Modules (DealHunterCouponEngine, SEOBlogPublisher with inline product cards,
TrueProfitAccountant, SalesForecaster, MonthlyReportBuilder).
   - Sprint 5: Build Node.js Orchestrator (orchestrator/src/server.js, cron_scheduler.js, WooCommerce REST API v3 client,
menu_reorder.js, smart_redirects.js, restock_notifier.js, cod_rto_guard.js, cart_recovery.js, Discord notifier & bot).
   - Sprint 6: Package WordPress plugin (wordpress-plugin/shopraven-bridge/) and mount the ShopRaven Control Room dashboard
(frontend/nnstore_ai_dashboard.html). Run full pytest and Jest verification suites.
```